

PATHFIND MENTORSHIP PLATFORM

Full Developer Implementation & Technical Specification Report (Frontend & Backend)

Project:	AmaliTech Capstone Internship — Product Family 2
Date:	September 11, 2026
Target Roles:	Frontend Developer (React + TypeScript) & Backend Developer (FastAPI)
Figma Reference:	https://www.figma.com/design/ml9aOY09sY4KW0hEalPFnw/Untitled

1. Executive Summary & Architecture Overview

This specification report establishes the technical implementation roadmap for completing the Pathfinder mentorship platform backend and frontend applications. It aligns the existing codebase with the finalized Figma User Interface design, project deliverables schedule, and seed data requirements.

The system architecture consists of a React 19 + TypeScript Single Page Application (SPA) communicating over RESTful JSON APIs with a FastAPI backend engine backed by a PostgreSQL relational database. Key modules include Auth & Session Restoration, Mentor Search & Discovery, Request Lifecycle Management, and Admin Verification Queue.

2. Backend Developer Specification & API Contracts

The backend developer must extend the database models, implement missing API endpoints, enforce role-based access control (RBAC), and add database foreign key relationships.

2.1 Database Models & Schema Extensions (backend/models.py)

```
from enum import Enum as PyEnum
from uuid import uuid4
from sqlalchemy import Column, Integer, String, Text, ForeignKey, Enum, DateTime
from sqlalchemy.orm import Mapped, mapped_column, relationship
from .database import Base

class VerificationStatus(str, PyEnum):
    PENDING_VERIFICATION = "pending_verification"
    VERIFIED = "verified"
```

REJECTED = "rejected"

class User(Base):

```
__tablename__ = "users"
id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
email: Mapped[str] = mapped_column(String(255), unique=True, index=True, nullable=False)
hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)
role: Mapped[str] = mapped_column(String(50), default="mentee", nullable=False) # "mentee", "mentor", "admin"
verification_status: Mapped[VerificationStatus] = mapped_column(
    Enum(VerificationStatus), default=VerificationStatus.VERIFIED, nullable=False
)
profile: Mapped["MentorProfile | None"] = relationship("MentorProfile", back_populates="user", uselist=False)
```

class MentorProfile(Base):

```
__tablename__ = "mentor_profiles"
id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
user_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), nullable=False, unique=True)
full_name: Mapped[str] = mapped_column(String(255), nullable=False)
job_title: Mapped[str] = mapped_column(String(255), nullable=False)
company: Mapped[str] = mapped_column(String(255), nullable=False)
years_of_experience: Mapped[int] = mapped_column(Integer, nullable=False)
bio: Mapped[str] = mapped_column(Text, nullable=False)
expertise_tags: Mapped[str] = mapped_column(String(500), nullable=False) # Comma-separated or JSON
availability: Mapped[str] = mapped_column(String(255), nullable=False)
avatar_url: Mapped[str | None] = mapped_column(String(500), nullable=True)

user: Mapped["User"] = relationship("User", back_populates="profile")
```

class MentorshipRequest(Base):

```
__tablename__ = "mentorship_requests"
id: Mapped[str] = mapped_column(String(36), primary_key=True, default=lambda: str(uuid4()))
mentee_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), nullable=False, index=True)
mentor_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), nullable=False, index=True)
request_type: Mapped[RequestType] = mapped_column(Enum(RequestType), nullable=False, index=True)
subject: Mapped[str] = mapped_column(String(255), nullable=False)
message: Mapped[str] = mapped_column(Text, nullable=False)
resume_url: Mapped[str | None] = mapped_column(String(500), nullable=True)
portfolio_url: Mapped[str | None] = mapped_column(String(500), nullable=True)
github_url: Mapped[str | None] = mapped_column(String(500), nullable=True)
response_message: Mapped[str | None] = mapped_column(Text, nullable=True)
status: Mapped[RequestStatus] = mapped_column(Enum(RequestStatus), default=RequestStatus.PENDING,
nullable=False)
```

2.2 Required Backend API Endpoints Implementation List

- **GET /auth/me:** Retrieves current authenticated user details (id, email, role, verification_status, profile) for frontend session restoration.

- **POST /auth/signup/mentor:** Handles multi-step mentor application, creates User & MentorProfile, sets verification_status = PENDING_VERIFICATION.
- **GET /mentors:** Mentor discovery search API with filtering params (?query=..., &expertise=..., &request_type=..., &verified_only=true).
- **GET /mentors/{id}:** Retrieves full profile details of a specific mentor for display on the Figma Mentor Detail modal.
- **PATCH /mentorship-requests/{id}/status:** Updates request status (ACCEPTED, DECLINED, COMPLETED) and attaches optional mentor response_message.
- **DELETE /mentorship-requests/{id}:** Allows mentees to cancel requests that are currently in PENDING state.
- **GET /admin/mentors/pending:** Lists all mentor applications pending verification (requires Admin RBAC guard).
- **POST /admin/mentors/{id}/approve:** Approve action: sets mentor's verification_status = VERIFIED and enables profile in directory.
- **POST /admin/mentors/{id}/reject:** Reject action: sets mentor's verification_status = REJECTED.

3. Frontend Developer Specification & Figma Screen Mapping

The frontend developer must build component layouts matching the Figma design frames, manage client-side state, enforce route protection, and integrate with backend REST APIs.

3.1 Screen-by-Screen Component Breakdown

Screen 1: Landing Page & Hero (/)

Contains Hero section, Search preview, Category Cards (CV Review, Portfolio, Career, Interview Prep), How It Works flow, and Header navigation.

Screen 2: Multi-Step Mentor Registration (/signup/mentor)

5-Step wizard: Step 1 (Account Credentials & Org Email), Step 2 (Title & Company), Step 3 (Skills & Request Types), Step 4 (Bio & Availability), Step 5 (Pending Review Notice).

Screen 3: Mentor Directory Canvas (/mentors)

Real-time search bar, Skill tag pills multi-select, Verified Mentor badge filter, Mentor Card Grid with avatar, title, skills, and Request CTA.

Screen 4: Mentor Detail Modal (/mentors/:id)

Bio paragraph, Company & Experience tags, Supported request types list, Availability schedule, and CTA to open Request Modal.

Screen 5: Mentorship Request Modal (/requests/new/:id)

Form with Mentor Header, Request Type selector, Subject input, Message textarea, and URL attachment fields (Resume URL, Portfolio URL, GitHub URL).

Screen 6: Mentee Dashboard (/dashboard/requests)

Tab filters (All, Pending, Accepted, Declined), Request Cards with Status Badges, Contact email reveal on Accepted, and Cancel Request button.

Screen 7: Mentor Dashboard (/mentor/dashboard)

Metrics summary cards, Incoming Request cards with mentee info & attachments, Accept button (reveals email), Decline modal with note textarea.

Screen 8: Admin Verification Queue (/admin/verification)

Pending Mentors Table showing Work Email, Company Domain (@company.com), Title, Date, and Approve / Reject action buttons.

4. Master API Contract Matrix

Summary mapping of frontend triggers to backend API endpoints:

HTTP Method	Endpoint Path	Auth Required	Key Payload / Query Parameters
GET	/auth/me	Yes	Header: Authorization: Bearer <token>
POST	/auth/signup	No	{ email, password, role }
POST	/auth/signup/mentor	No	{ full_name, email, password, job_title, company, expertise }
POST	/auth/signin	No	{ email, password }
GET	/mentors	No	? query=...&expertise=...&request_type=...&verified_only=true
GET	/mentors/{id}	No	Path Param: {id}
POST	/mentorship-requests	Yes	{ mentor_id, subject, message, resume_url, portfolio_url }
GET	/mentorship-requests	Yes	Header: Authorization: Bearer <token>
PATCH	/mentorship-requests/	Yes	{ status:

	{id}/status		'accepted' 'declined', response_message }
DELETE	/mentorship-requests/{id}	Yes	Path Param: {id}
GET	/admin/mentors/pending	Yes (Admin)	Header: Authorization: Bearer <admin_token>
POST	/admin/mentors/{id}/ approve	Yes (Admin)	Path Param: {id}

5. Development Task Roadmap & Deliverables

5.1 Backend Developer Deliverables

- Create MentorProfile database model and run initial migration.
- Add Foreign Key constraints linking mentorship_requests to users.id.
- Implement GET /auth/me for user session restoration.
- Implement POST /auth/signup/mentor for multi-step mentor application.
- Implement GET /mentors search API with real-time text query and skill filtering.
- Implement PATCH /mentorship-requests/{id}/status with mentor response notes.
- Implement Admin Verification Queue endpoints (GET /admin/mentors/pending & POST /admin/mentors/{id}/approve).

5.2 Frontend Developer Deliverables

- Build Multi-Step Mentor Registration Form (/signup/mentor) with validation.
- Build Mentor Directory page (/mentors) with real-time search bar and skill pills.
- Build Mentor Profile Detail modal (/mentors/:id).
- Add Resume, Portfolio, and GitHub URL input fields to Request Submission Modal.
- Build Mentor Dashboard incoming request cards with Accept (reveals email) and Decline (note textarea) action modals.
- Build Admin Pending Verification Queue table (/admin/verification) with Approve and Reject action buttons.